

Học Sâu (Deep Learning)

Chương 14: Phân loại Văn bản (Text Classification & NLP)

Đại học Đông Á

August 14, 2026

Nội dung chương

1. Giới thiệu Xử lý Ngôn ngữ Tự nhiên (NLP)
2. Chuỗi Tiền xử lý Văn bản (Text Pipeline)
3. Phương pháp Biểu diễn Túi từ (Bag-of-Words)
4. Mô hình Trình tự và Word Embeddings - Trái tim của NLP Hiện đại
5. Tổng kết

Lịch sử và Bản chất của NLP

- Ngôn ngữ tự nhiên (Tiếng Việt, Tiếng Anh) khác biệt hoàn toàn với ngôn ngữ lập trình (C++, Python). Nó luôn biến động, mập mờ, phức tạp và phá vỡ quy tắc ngữ pháp của chính nó.
- **Natural Language Processing (NLP)** là lĩnh vực giao thoa giữa AI và Ngôn ngữ học nhằm giúp máy tính có thể "đọc", "hiểu" và "giao tiếp" bằng ngôn ngữ con người.
- Những năm 1950 - 1990: Thống trị bởi các bộ quy tắc gõ tay (Rule-based / Symbolic AI). Rất cứng nhắc và dễ thất bại.
- Cuối 1990 - Nay: Dịch chuyển sang **Machine Learning & Statistical NLP** (Mô hình thống kê tự học luật từ dữ liệu).

Các Bài toán Cốt lõi của NLP

NLP xử lý một lượng lớn các ứng dụng trong đời sống hiện đại:

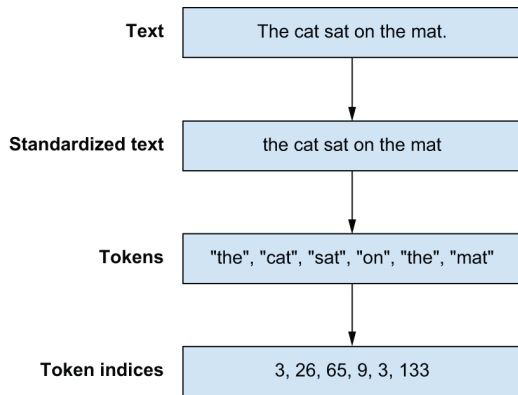
- **Text Classification (Phân loại văn bản):** Email này có phải là Thư rác (Spam) không? Bình luận này mang cảm xúc Tiêu cực hay Tích cực (Sentiment Analysis)?
- **Machine Translation (Dịch máy):** Dịch một câu tiếng Anh sang tiếng Việt sao cho hợp ngữ cảnh nhất (Google Translate).
- **Language Modeling (Mô hình ngôn ngữ):** Cho trước một đoạn văn, hãy dự đoán từ tiếp theo sẽ là gì? (Nền tảng của ChatGPT).

Tại sao cần Tiền xử lý Văn bản?

- Máy tính không hiểu được các ký tự (Char) hay các từ (Word). Mạng Nơ-ron chỉ hoạt động trên các Ma trận và Tensor gồm các **Con số (Numbers)**.
- Do đó, ta phải Vector hóa (Vectorize) văn bản, tức là biến đổi chuỗi văn bản thô thành một chuỗi các con số hoặc một ma trận có cấu trúc.
- Quá trình này không thể làm tùy tiện mà phải tuân theo một Chuỗi xử lý (Text Pipeline) nghiêm ngặt gồm 3 bước.

Text Pipeline: 3 Bước Bắt đầu bất dịch

- 1 **Standardization (Chuẩn hóa):** Đưa văn bản về một chuẩn chung. Ví dụ: Chuyển toàn bộ thành chữ thường (lower case), xóa bỏ các dấu câu chấm phẩy dư thừa (Punctuation removal).
- 2 **Tokenization (Tách từ):** Chẻ câu văn bản thành các đơn vị rời rạc gọi là "Tokens" (Có thể là Từ, Ký tự, hoặc Cụm từ).
- 3 **Indexing (Đánh chỉ mục):** Chuyển mỗi Token thành một con số duy nhất bằng cách xây dựng Bộ Từ điển (Vocabulary) cho toàn bộ tập dữ liệu.



Hình 14.1: Tổng quan Quy trình xử lý Văn bản

Tokenization: Tách ký tự và Tách từ bằng Regex

Trong Python, regex là công cụ mạnh mẽ nhất để nhận diện mẫu.

```
1 import regex as re
2
3 # 1. Split into characters
4 def split_chars(text):
5     return re.findall(r".", text)
6
7 chars = split_chars("The quick brown fox.")
8 # ["T", "h", "e", " ", "q", "u", "i", "c", "k", " ", "...]
9
10 # 2. Split into words and punctuation
11 def split_words(text):
12     # Matches consecutive word characters OR punctuation marks
13     return re.findall(r"[\w]+|[.,!?!;]", text)
14
15 words = split_words("The quick brown fox jumped over the dog.")
16 # ["The", "quick", "brown", "fox", "jumped", "over", "the", "dog", "."]
```

Indexing: Xây dựng Từ điển (Vocabulary)

Kỹ thuật mạnh mẽ nhất là xây dựng ánh xạ từ các mã thông báo cụ thể đến các chỉ mục bằng từ điển (Dictionary / Vocabulary).

```
1 # Mapping tokens to unique integer indices
2 vocabulary = {
3     "[UNK]": 0, # Unknown token placeholder
4     "the": 1,
5     "quick": 2,
6     "brown": 3,
7     "fox": 4,
8     "jumped": 5,
9     "over": 6,
10    "dog": 7,
11    ".": 8,
12 }
13 words = split_words("The quick brown fox jumped over the lazy dog.")
14
15 # Index the tokens. "lazy" becomes 0 because it's not in vocab.
16 indices = [vocabulary.get(word.lower(), 0) for word in words]
17 print(indices)
18 # Output: [1, 2, 3, 4, 5, 6, 1, 0, 7, 8]
```


Nhược điểm của Tách Ký tự và Tách Từ

- **Cấp độ Ký tự (Char-level):** Từ điển siêu nhỏ (Khoảng 60-100 ký tự ASCII/Unicode) nhưng mỗi câu bị chẻ thành hàng trăm tokens nhỏ lẻ → Mất hết ý nghĩa logic, mô hình rất khó học.
- **Cấp độ Từ (Word-level):** Giữ nguyên được ý nghĩa ngữ nghĩa trọn vẹn, chuỗi ngắn gọn. Nhưng từ điển sẽ phình to ra khổng lồ (hàng trăm ngàn từ). Những từ hiếm (Rare words) hoặc sai chính tả sẽ bị gom vào [UNK].
- **Giải pháp dung hòa:** Mã hóa từ phụ (Sub-word Tokenization).

Mã hóa từ phụ (Sub-word Tokenization) và BPE

- **Mục tiêu:** Giữ từ điển nhỏ như Char-level nhưng độ dài chuỗi ngắn như Word-level.
- **Thuật toán BPE (Byte-Pair Encoding):** Bắt đầu với từng ký tự đơn lẻ. Lặp lại việc **gộp (merge)** 2 ký hiệu xuất hiện cùng nhau nhiều nhất thành một ký hiệu mới.
- Ví dụ gộp BPE:
 - "l" + "o" → "lo"
 - "lo" + "w" → "low"
- Đây là thuật toán đứng đằng sau các mô hình khổng lồ như GPT-3, ChatGPT và BERT.

Tự động hóa với Keras TextVectorization

Thay vì tự viết hàm, Keras cung cấp layer TextVectorization siêu việt. Khác với mã thông thường chạy trên CPU, lớp này có thể lồng thẳng vào Model và hoạt động trên GPU/TPU.

```
1 from keras import layers
2
3 # Configure the layer with a maximum vocabulary size
4 text_vectorization = layers.TextVectorization(
5     max_tokens=20000,
6     output_mode="int", # Return sequence of token indices
7     split="whitespace",
8     standardize="lower_and_strip_punctuation"
9 )
10
11 # Build vocabulary directly from a dataset
12 text_vectorization.adapt(text_dataset)
13
14 # Encode a new string
15 encoded = text_vectorization([["A new text string"]])
```

Phương pháp Túi từ (Bag-of-Words)

- Lịch sử NLP bắt đầu với các mô hình **Set-based (Dựa trên Tập hợp)**.
- Thuật toán Bag-of-Words: Chỉ quan tâm từ X xuất hiện **có mặt hay vắng mặt** trong văn bản, hoàn toàn vứt bỏ trật tự thời gian.
- **Hạn chế chí mạng**: Hai câu "Chó cắn người" và "Người cắn chó" sẽ có chung một vector biểu diễn y hệt nhau, dù ý nghĩa hoàn toàn trái ngược.
- Để vượt vát lại thứ tự cục bộ, người ta dùng khái niệm **N-grams** (Nhóm 2-3 từ liên tiếp nhau). Ví dụ Bi-grams: "Chó cắn", "cắn người".

Thuật toán TF-IDF (Term Frequency-Inverse Document Frequency)

- Khi dùng Bag-of-Words, các từ như "the", "a", "is", "và", "nhưng" sẽ xuất hiện cực nhiều nhưng mang rất ít giá trị nội dung. Các từ khóa chuyên ngành lại xuất hiện ít nhưng rất giá trị.
- **TF (Term Frequency):** Tần suất từ t xuất hiện trong văn bản hiện tại (Càng nhiều càng tốt).
- **IDF (Inverse Document Frequency):** Nghịch đảo tần suất của từ t trên toàn bộ kho tài liệu (Nói lên độ hiếm có của từ t).

$$\text{TF-IDF} = TF \times \log \left(\frac{N}{df_t} \right)$$

- Nhờ TF-IDF, các từ hiếm sẽ được tăng trọng số, trong khi các từ phổ biến vô nghĩa sẽ bị phạt (đưa về 0).

Áp dụng Bag-of-Words cho tập Dữ liệu IMDb

Sử dụng `output_mode="multi_hot"` để biến mỗi câu văn bản thành một mảng Zeroes độ dài 20,000. Nếu từ có index i xuất hiện trong câu, giá trị tại vị trí i sẽ là 1.

```
1 max_tokens = 20000
2
3 # Create multi-hot encoding TextVectorization
4 text_vectorization = layers.TextVectorization(
5     max_tokens=max_tokens,
6     split="whitespace",
7     output_mode="multi_hot", # Crucial for Bag-of-words
8 )
9
10 # Learn the vocabulary
11 text_vectorization.adapt(train_ds_no_labels)
12
13 # Map the datasets
14 bag_of_words_train_ds = train_ds.map(
15     lambda x, y: (text_vectorization(x), y), num_parallel_calls=8
16 )
17 # A batch of 32 outputs has shape (32, 20000)
```

Xây dựng Linear Classifier cho Bag-of-Words

Do mỗi đầu vào bây giờ là một `multi_hot` vector 20,000 chiều (chứa nhiều số 0 và số 1), ta chỉ cần sử dụng duy nhất một lớp Dense để tính tổng trọng số.

```
1 import keras
2
3 def build_linear_classifier(max_tokens, name):
4     inputs = keras.Input(shape=(max_tokens,))
5     # A single dense layer for binary classification
6     outputs = layers.Dense(1, activation="sigmoid")(inputs)
7     model = keras.Model(inputs, outputs, name=name)
8
9     model.compile(
10         optimizer="adam",
11         loss="binary_crossentropy",
12         metrics=["accuracy"],
13     )
14     return model
15
16 model = build_linear_classifier(20000, "bag_of_words_classifier")
```

Lưu ý: Mạng này chỉ có 20,001 tham số (Rất nhỏ so với các mạng nơ-ron khác).

Tiền xử lý hiệu quả với CPU và GPU

- Quá trình Tiền xử lý văn bản (Regex, Tách từ, Xây dựng từ điển) **bắt buộc phải chạy trên CPU**. GPU không biết xử lý chuỗi ký tự thô.
- Nếu CPU xử lý chậm, GPU đắt tiền sẽ phải "ngồi chờ" (Bottleneck).
- **Giải pháp:** Keras cung cấp `tf.data.Dataset` cho phép xử lý văn bản đa luồng (Multi-threading) bằng cách chạy song song `.map()` trên nhiều lõi CPU, cung cấp dữ liệu số liên tục cho GPU học tập.

Huấn luyện mô hình Bag-of-Words cơ bản

Mặc dù là mạng nông, nó huấn luyện nhanh và cực kỳ hiệu quả.

```
1 # Stop early if validation loss gets worse
2 early_stopping = keras.callbacks.EarlyStopping(
3     monitor="val_loss",
4     restore_best_weights=True,
5     patience=2,
6 )
7
8 history = model.fit(
9     bag_of_words_train_ds,
10    validation_data=bag_of_words_val_ds,
11    epochs=10,
12    callbacks=[early_stopping],
13 )
14
15 # Test Accuracy
16 test_loss, test_acc = model.evaluate(bag_of_words_test_ds)
17 print(f"Test Accuracy: {test_acc:.4f}") # Output: 0.8838
```

Độ chính xác đạt **88.38%** - rất đáng nể với thuật toán ném bỏ toàn bộ thứ tự từ!

Tối ưu hóa: Bigram Model (Lưu vết thứ tự cục bộ)

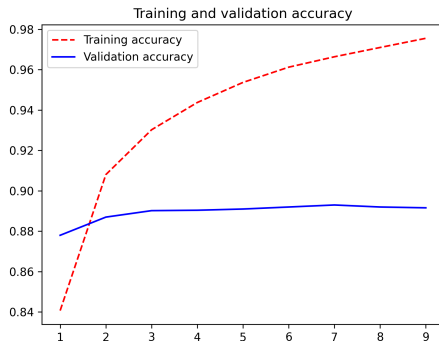
Ta có thể nâng cấp Bag-of-Words bằng cách sử dụng các cặp 2 từ (Bigrams). Cụm từ "Hoa Kỳ" mang nghĩa khác hoàn toàn chữ "Hoa" và chữ "Kỳ" tách rời.

```
1 max_tokens = 30000 # Increase vocab size to hold bigrams
2 text_vectorization = layers.TextVectorization(
3     max_tokens=max_tokens,
4     split="whitespace",
5     output_mode="multi_hot",
6     ngrams=2, # Use both Unigrams and Bigrams
7 )
8 text_vectorization.adapt(train_ds_no_labels)
9
10 # Example vocab learned: ["in a", "most", "him", "dont", "it was", "one of"]
```

Kết quả: Sau khi train lại, Accuracy đẩy lên **90.11%**. Nó cực kỳ nhanh nhưng nếu cố tăng n-gram lên quá lớn (ví dụ 4-gram) thì bộ nhớ sẽ cạn kiệt theo cấp số nhân.

Hiệu suất của Bag-of-Words

- Thử nghiệm phân loại cảm xúc tích cực / tiêu cực trên tập dữ liệu IMDB (Đánh giá phim).
- Kết hợp Bigrams (N-grams) và mô hình Dense.
- **Kết quả:** Đạt độ chính xác (Accuracy) lên tới hơn 89-90% trên tập Validation.
- Đối với phân loại đơn giản, Bag-of-Words vẫn là một Baseline cực kỳ đáng gờm vì tính nhẹ và cực nhanh của nó.



Hình 14.2: Độ chính xác của Bag-of-Words

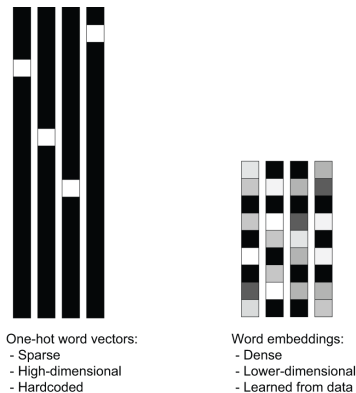
Mô hình Trình tự (Sequence Models)

Để học tập chuỗi sâu sắc (RNN, LSTM, Transformer), đầu vào bắt buộc phải duy trì theo đúng thời gian tuyến tính. Các câu không bằng nhau cần được Đệm (Padding) hoặc Cắt bỏ (Truncating).

```
1 max_length = 600 # Force every sequence to be exactly 600 tokens long
2 max_tokens = 30000
3
4 text_vectorization = layers.TextVectorization(
5     max_tokens=max_tokens,
6     split="whitespace",
7     output_mode="int", # Outputs an integer sequence, not a multi_hot set
8     output_sequence_length=max_length, # Pad with zeroes if short, cut if long
9 )
10 text_vectorization.adapt(train_ds_no_labels)
11
12 # Example Output:
13 # "the slow brown badger" -> ["the", "slow", "brown", "badger", "[PAD]", "[PAD]"]
14 # -> [1, 255, 331, 840, 0, 0]
```

Khuyết điểm của One-hot Encoding

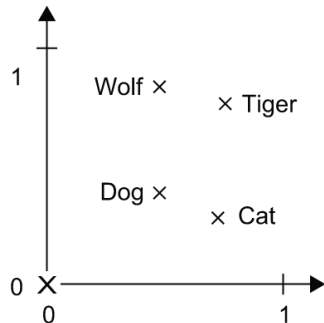
- Trước đây, mỗi từ được biểu diễn bằng One-hot Vector: Một véc-tơ chứa toàn số 0, chỉ có duy nhất một số 1.
- **Tính chất rời rạc:** Mọi từ đều độc lập. Khoảng cách toán học (Dot product) giữa từ "Chó" và "Mèo" bằng chính xác khoảng cách giữa "Chó" và "Cái bàn" (Bằng 0).
- Mô hình không học được tính chất tương đồng ngữ nghĩa.



Hình 14.3: Biểu diễn Sparse vs Dense

Sự trỗi dậy của Word Embeddings (Nhúng từ)

- **Word Embeddings** là biểu diễn Dày đặc (Dense), Liên tục (Continuous) mang số thực. Thay vì 10,000 chiều 0, 1, ta chỉ dùng 256 chiều số thực.
- Những từ có **Ngữ cảnh giống nhau** sẽ nằm **Gần nhau** trong không gian hình học đa chiều.
- Từ "Dog" và "Wolf" sẽ hội tụ về một góc, khác hoàn toàn với góc của "Car" hay "Bicycle".



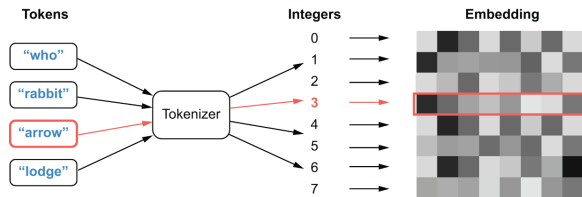
Hình 14.4: Không gian Nhúng (Embedding Space)

Dại số tuyến tính trong Không gian Ngữ nghĩa

- Nhờ cấu trúc Dense Vector, mạng Nơ-ron có thể làm **Toán học trên ngôn ngữ**.
- Phép cộng trừ véc-tơ phản ánh chính xác các khái niệm logic của con người:
 - $\text{Véc-tơ(Vua)} - \text{Véc-tơ(Nam)} + \text{Véc-tơ(Nữ)} = \text{Véc-tơ(Nữ hoàng)}$
 - $\text{Véc-tơ(Paris)} - \text{Véc-tơ(Pháp)} + \text{Véc-tơ(Đức)} = \text{Véc-tơ(Berlin)}$
- Đây là bước nhảy vọt quan trọng nhất trong lịch sử NLP.

Lớp Embedding (Embedding Layer) trong Keras

- Trong mô hình học sâu, lớp Embedding đóng vai trò như một Cuốn từ điển (Dictionary Lookup).
- Đầu vào là một Chỉ mục số nguyên (VD: Từ "Apple" có index là 12).
- Đầu ra là Véc-tơ mang ngữ nghĩa của từ đó tương ứng với Hàng thứ 12 trong Ma trận trọng số W .
- Ma trận W này được Cập nhật (Huấn luyện) liên tục qua thuật toán Backpropagation.



Hình 14.5: Cơ chế hoạt động của Lớp Embedding

Sử dụng layers.Embedding cho mô hình LSTM

Thêm layers.Embedding trước mạng LSTM để thay đổi dữ liệu từ các Index thành các Không gian Dense liên tục chứa biểu diễn ngữ nghĩa.

```
1 inputs = keras.Input(shape=(max_length,))
2
3 # Embedding Layer translates integer sequences to dense vectors
4 # Here, each word becomes a 256-dimensional vector
5 x = layers.Embedding(
6     input_dim=max_tokens,
7     output_dim=256,
8     mask_zero=True, # Ignore the zeroes generated by padding
9 )(inputs)
10
11 # Now, we can pipe the vectors to a recurrent network
12 x = layers.Bidirectional(layers.LSTM(32))(x)
13 x = layers.Dropout(0.5)(x)
14 outputs = layers.Dense(1, activation="sigmoid")(x)
15
16 model = keras.Model(inputs, outputs)
```

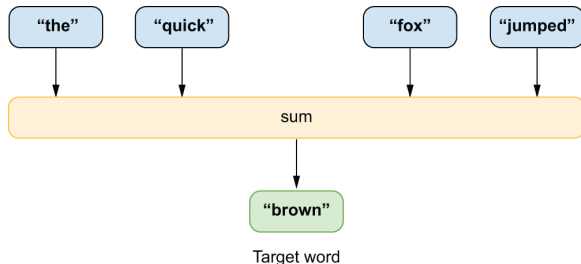
Vấn đề Dữ liệu Nhãn (Labeled Data) bị hạn chế

- Lớp Embedding giảm số tham số khổng lồ của một mạng NLP (Từ 15 triệu xuống 2 triệu), mô hình tính toán cực nhanh.
- **Hạn chế:** Trong ví dụ LSTM trước, độ chính xác bị kẹt ở mức 84%. Tệ hơn mô hình Bag-of-Words cơ bản.
- **Lý do:** Mạng chứa Embedding + LSTM quá lớn và mạnh. Tập dữ liệu 20,000 review phim là quá ít để mô hình học được Ngữ nghĩa của thế giới ngôn ngữ rộng lớn. Mô hình bị Overfit nặng (Học thuộc lòng tập train).
- **Giải pháp:** Pre-training (Tiền huấn luyện) không giám sát.

Kiến trúc học Nhúng: Word2Vec và CBOW

Làm sao có được bộ trọng số Embedding xin từ đầu? (Pre-trained Word Embeddings).

- Thuật toán ****Word2Vec**** (Do Tomas Mikolov - Google giới thiệu 2013).
- Sử dụng mô hình ****CBOW** (Continuous Bag-of-Words)**.
- Nguyên lý: Ép mô hình **Dự đoán từ ở giữa** dựa trên các từ ngữ cảnh xung quanh nó (Unsupervised). Trọng số học được sẽ cấu trúc Không gian Vector hợp lý nhất.



Hình 14.6: Mô hình dự đoán ngữ cảnh CBOW

Chuẩn bị dữ liệu Cửa sổ (Context Window) cho CBOW

Dùng TensorFlow API để cắt nhỏ 1 câu dài thành các túi ngữ cảnh 8 chữ chứa từ mục tiêu bị ẩn.

```
1 import tensorflow as tf
2
3 context_size = 4 # 4 words left, 4 words right
4 window_size = 9 # Total 9 words in window
5
6 # Function to extract label (center word) from context
7 def split_label(window):
8     left = window[:context_size]
9     right = window[context_size + 1 :]
10    bag = tf.concat((left, right), axis=0) # Shape: 8 context words
11    label = window[4] # The word to predict
12    return bag, label
13
14 # Create sliding windows from text dataset
15 # [the, quick, brown, FOX, jumped, over, the, lazy] => Label: FOX
16 dataset = dataset.interleave(window_data)
17 dataset = dataset.map(split_label)
```

Xây dựng mô hình CBOW dự đoán nhãn

Một mạng vô cùng nông kết hợp Embedding và Pool Average. Sau đó huấn luyện trên hàng triệu Text tự do. Vector Embedding thu được sẽ mang thông tin cấu trúc Ngôn ngữ tuyệt vời để cấy vào LSTM.

```
1 hidden_dim = 64
2 inputs = keras.Input(shape=(2 * context_size,)) # 8 inputs
3
4 # The core embedding layer we want to train
5 cbow_embedding = layers.Embedding(max_tokens, hidden_dim)
6 x = cbow_embedding(inputs) # Shape: (8, 64)
7
8 # Average all 8 vectors into a single 64-dim vector
9 x = layers.GlobalAveragePooling1D()(x)
10
11 # Predict the word out of the entire dictionary
12 outputs = layers.Dense(max_tokens, activation="sigmoid")(x)
13
14 cbow_model = keras.Model(inputs, outputs)
15 cbow_model.compile(loss="sparse_categorical_crossentropy", optimizer="adam")
```

Sử dụng Pre-trained Embeddings

Thay vì train Embedding từ đầu trên 20k review, ta có thể tải Embedding đã được train trên toàn bộ Wikipedia (GloVe, Word2Vec) và cắm vào layer.

```
1 import numpy as np
2
3 # Load pretrained weights into the Embedding layer
4 embedding_layer = layers.Embedding(
5     max_tokens,
6     256,
7     embeddings_initializer=keras.initializers.Constant(pretrained_matrix),
8     trainable=False, # Freeze the layer!
9 )
10
11 # Build LSTM model
12 inputs = keras.Input(shape=(max_length,))
13 x = embedding_layer(inputs)
14 x = layers.Bidirectional(layers.LSTM(32))(x)
15 outputs = layers.Dense(1, activation="sigmoid")(x)
```

Sự lựa chọn giữa Set-based và Sequence-based

- **Bag-of-Words (Sets):** Sử dụng khi cần train cực nhanh, hoặc chỉ quan tâm đến các **từ khóa** trong câu hơn là ngữ cảnh sâu xa (Phân tích cảm xúc đơn giản, Lọc Spam).
- **Sequence Models (RNN, LSTM, Transformer):** Sử dụng khi **trật tự từ** quyết định ý nghĩa câu (Dịch máy, Tạo văn bản, Trả lời câu hỏi). Yêu cầu lượng lớn dữ liệu để không bị Overfitting.
- Nếu tập dữ liệu $\leq 100,000$ mẫu, dùng Set-based (Bigrams) kết hợp mạng Dense nhỏ là lựa chọn khôn ngoan nhất.

Tổng kết Chương 14

- Máy tính không hiểu văn bản thô. Cần có ****Text Pipeline**** (Chuẩn hóa → Tách từ → Đánh chỉ mục).
- ****Bag-of-Words / N-grams + TF-IDF**** Đơn giản, cực nhẹ, hiệu quả cao trong phân loại văn bản nhanh, nhưng không nhớ được cấu trúc thời gian của câu.
- ****Word Embeddings (Nhúng từ):**** Là bước đệm vĩ đại của NLP. Chuyển từ sang Không gian Vector Ngữ nghĩa liên tục đa chiều (Toán học hóa ngôn ngữ).
- Thuật toán như Word2Vec (CBOW, Skip-gram) giúp khai thác triệt để tài nguyên text khổng lồ trên Internet mà không cần nhãn (Unsupervised learning).